

Homework 06 - Static, Modifiers, Abstract Class And Interface

Topics Covered

- Local variable
- Instance variable
- Static variable
- Static method
- Static block
- Why `main` method is static
- Access modifiers
- Non-access modifiers
- `final` keyword
- Abstract class
- Abstract method
- Constructor in abstract class
- Interface
- Multiple inheritance using interface
- Abstract class vs interface

Instructions

1. Write every program in a separate Java file.
2. First write the expected output, then run the program.
3. Use meaningful class, method, and variable names.
4. Use comments only where explanation is really needed.
5. For every error-fixing question, write the reason before correcting the code.
6. After coding questions, write 2-3 lines explaining which concept is used.

Part A: Short Answer Questions

1. What is a local variable?
2. Where is a local variable declared?
3. Does a local variable get a default value?
4. What is an instance variable?
5. Why is an instance variable called object-level variable?
6. What is a static variable?
7. Why is a static variable called class-level variable?
8. What is the difference between instance variable and static variable?
9. How should we access a static variable?
10. What is a static method?

11. Can a static method directly access an instance variable? Why?
12. Can a static method directly access a static variable?
13. Why is the `main` method static in Java?
14. What is a static block?
15. How many times does a static block execute?
16. What is the difference between static block and constructor?
17. What is an access modifier?
18. Name all four access levels in Java.
19. Is `default` an access modifier keyword?
20. What is the use of `private` ?
21. What is the use of `protected` ?
22. What is a non-access modifier?
23. What is the use of `final` variable?
24. What is the use of `final` method?
25. What is the use of `final` class?
26. What is an abstract class?
27. Can we create object of abstract class directly?
28. What is an abstract method?
29. Can an abstract class have constructor?
30. What is an interface?
31. Why do we use interface?
32. Which keyword is used to implement an interface?
33. Can one class implement multiple interfaces?
34. What is the difference between abstract class and interface?
35. When should we use abstract class and when should we use interface?

Part B: Find The Output

Question 1

```
class Student {
    String name;
    int age;
}

public class Main {
    public static void main(String[] args) {
        Student student = new Student();

        System.out.println(student.name);
        System.out.println(student.age);
    }
}
```

Question 2

```

class Counter {
    int instanceCount = 0;
    static int staticCount = 0;

    Counter() {
        instanceCount++;
        staticCount++;

        System.out.println(instanceCount + " " + staticCount);
    }
}

public class Main {
    public static void main(String[] args) {
        new Counter();
        new Counter();
        new Counter();
    }
}

```

Question 3

```

class Student {
    String name;
    static String instituteName = "Genie Academy";
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student();

        s1.name = "Rahul";
        s2.name = "Priya";

        Student.instituteName = "Genie Ashwani";

        System.out.println(s1.name + " " + s1.instituteName);
        System.out.println(s2.name + " " + s2.instituteName);
    }
}

```

Question 4

```
class Demo {
    static {
        System.out.println("Static block");
    }

    Demo() {
        System.out.println("Constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Demo d1 = new Demo();
        Demo d2 = new Demo();
    }
}
```

Question 5

```
class Calculator {
    static int add(int a, int b) {
        return a + b;
    }

    static int multiply(int a, int b) {
        return a * b;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println(Calculator.add(10, 20));
        System.out.println(Calculator.multiply(5, 4));
    }
}
```

Question 6

```
abstract class User {
    User() {
        System.out.println("User constructor");
    }
}

class Student extends User {
    Student() {
        System.out.println("Student constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        Student student = new Student();
    }
}
```

Question 7

```
abstract class Payment {
    abstract void pay();

    void printReceipt() {
        System.out.println("Receipt printed");
    }
}

class UpiPayment extends Payment {
    void pay() {
        System.out.println("Paid using UPI");
    }
}

public class Main {
    public static void main(String[] args) {
        Payment payment = new UpiPayment();
        payment.pay();
        payment.printReceipt();
    }
}
```

Question 8

```
interface Printable {
    void print();
}

interface Scannable {
    void scan();
}

class PrinterMachine implements Printable, Scannable {
    public void print() {
        System.out.println("Printing");
    }

    public void scan() {
        System.out.println("Scanning");
    }
}

public class Main {
    public static void main(String[] args) {
        PrinterMachine machine = new PrinterMachine();
        machine.print();
        machine.scan();
    }
}
```

Part C: Find And Fix The Error

Question 1

```
public class Main {
    public static void main(String[] args) {
        int age;
        System.out.println(age);
    }
}
```

Write:

1. What is the error?
2. Why does it happen?
3. Correct the code.

Question 2

```
class Student {
    String name = "Rahul";

    static void showName() {
        System.out.println(name);
    }
}
```

Write:

1. What is the error?
2. Why cannot static method directly access `name` ?
3. Correct the code by passing object reference.

Question 3

```
class BankAccount {
    public double balance;
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.balance = -50000;
        System.out.println(account.balance);
    }
}
```

Write:

1. What is the design problem?
2. Which modifier can protect `balance` ?
3. Rewrite using private variable and public methods.

Question 4

```
final class Parent {
}

class Child extends Parent {
}
```

Write:

1. What is the error?
2. Why is inheritance not allowed here?

Question 5

```

class Parent {
    final void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child");
    }
}

```

Write:

1. What is the error?
2. What does final method mean?

Question 6

```

abstract class Vehicle {
    abstract void start();
}

class Car extends Vehicle {
}

```

Write:

1. What is the error?
2. Why must Car implement start() ?
3. Correct the code.

Question 7

```

interface Payment {
    void pay();
}

class UpiPayment implements Payment {
    void pay() {
        System.out.println("Paid using UPI");
    }
}

```

Write:

1. What is the error?
2. Why should the implemented method be public ?
3. Correct the code.

Question 8

```
abstract class Shape {  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Shape shape = new Shape();  
    }  
}
```

Write:

1. What is the error?
2. Can we create object of abstract class directly?
3. How can we use abstract class reference correctly?

Part D: Coding Practice

Program 1: Local And Instance Variable

Create a `Student` class with:

- instance variable `name`
- instance variable `marks`
- method `showResult()`

Inside `showResult()`, create a local variable `result`.

If marks are greater than or equal to 40, store "Pass" in `result`, otherwise store "Fail".

Print student name, marks, and result.

Program 2: Static Institute Name

Create a `Student` class with:

- instance variable `rollNumber`
- instance variable `name`
- static variable `instituteName`
- method `showDetails()`

Create 3 student objects and print details.

Change institute name using class name and print details again.

Program 3: Object Counter

Create a `Student` class with:

- static variable `studentCount`
- constructor that increases `studentCount`
- static method `showTotalStudents()`

Create 5 student objects and print total students.

Program 4: Utility Calculator

Create a `Calculator` class with static methods:

- `add(int a, int b)`
- `subtract(int a, int b)`
- `multiply(int a, int b)`
- `divide(int a, int b)`

Call all methods using class name.

Program 5: Static Block

Create a class `DatabaseConfig` with:

- static variable `databaseName`
- static block that initializes `databaseName`
- static method `showDatabaseName()`

Call `showDatabaseName()` from main method.

Program 6: Encapsulation With Access Modifier

Create a `BankAccount` class with:

- private variable `accountNumber`
- private variable `balance`
- constructor
- public method `deposit(double amount)`
- public method `withdraw(double amount)`
- public method `getBalance()`

Rules:

- Deposit only positive amount.
- Withdraw only if balance is enough.
- Balance should not be directly accessible outside the class.

Program 7: Final Variable

Create a class `Exam` with:

- final variable `MAX_MARKS = 100`
- method `showMaxMarks()`

Try to change `MAX_MARKS` and write the error reason in comments.

Program 8: Abstract Class

Create an abstract class `Shape` with:

- abstract method `calculateArea()`
- normal method `showMessage()`

Create child classes:

- `Circle`
- `Rectangle`

Implement `calculateArea()` in both child classes.

Program 9: Constructor In Abstract Class

Create an abstract class `User` with:

- instance variable `name`
- constructor to initialize `name`
- normal method `showName()`

Create child class `Student` with:

- instance variable `course`
- constructor using `super(name)`
- method `showStudentDetails()`

Create object of `Student` and print details.

Program 10: Interface

Create an interface `NotificationService` with:

```
void send(String message);
```

Create two classes:

- `EmailService`
- `SmsService`

Both classes should implement `NotificationService`.

In main method, create interface reference and call both implementations.

Program 11: Multiple Interfaces

Create two interfaces:

```
Printable
- print()

Scannable
- scan()
```

Create class `PrinterMachine` that implements both interfaces.

Call both methods from main method.

Program 12: Payment System

Create:

```
interface PaymentGateway
- pay(double amount)

abstract class BasePayment
- static appName
- transactionId
- constructor
- printReceipt()

class UpiPayment extends BasePayment implements PaymentGateway
- upiId
- constructor
- pay(double amount)

class CardPayment extends BasePayment implements PaymentGateway
- cardNumber
- constructor
- pay(double amount)
```

In main method:

1. Create one UPI payment object.
2. Create one card payment object.
3. Use `PaymentGateway` reference.
4. Call `pay()` method.

Part E: Concept Mapping

Complete the table:

Situation	Concept Used
Variable declared inside method	
Variable belongs to every object separately	
Common variable shared by all objects	
Method called using class name	
Block runs once when class loads	
JVM calls main without object	
Data accessible only inside same class	
Value cannot be changed	
Method cannot be overridden	
Class cannot be inherited	
Class contains incomplete method	
Method without body	
Contract followed by class	
One class implements two capabilities	

Part F: Difference Questions

Write differences:

1. Local variable vs Instance variable
2. Instance variable vs Static variable
3. Static method vs Instance method
4. Static block vs Constructor
5. `private` vs `public`
6. Access modifier vs Non-access modifier
7. Final variable vs Final method vs Final class
8. Abstract class vs Concrete class
9. Abstract method vs Normal method
10. Abstract class vs Interface
11. `extends` vs `implements`
12. Interface inheritance vs Class inheritance

Part G: Real-World Thinking Questions

1. In a school application, which data should be instance variable and which data can be static?
2. Why should bank account balance be private?
3. Why can a calculator utility method be static?

4. Why should some rules be made final?
5. Why can `Payment` be an interface?
6. Why can `BasePayment` be an abstract class?
7. Why does Java allow multiple interfaces but not multiple classes?
8. In backend projects, why do we usually code using interface reference?

Submission Checklist

Before submitting, check:

- Did you explain local, instance, and static variables clearly?
- Did you access static members using class name?
- Did you explain why `main` is static?
- Did you use `private` fields where required?
- Did you correctly use `final` ?
- Did every abstract method get implemented in child class?
- Did every interface method get implemented as `public` ?
- Did you write differences in your own words?
- Did you write expected output before running programs?

